SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 14.5

Program	***: B. Tech (CSE)***
Specialization :	
Course Title	***: AI Assisted coding***
Course Code	***:*** 23CS002PC304
Semester	***: II***
Academic Session	***: 2025-2026***
Name of Student	*: I.Sathwik*
Enrollment No.	***: 2403A51L03***
Batch No.	***: 51***
Date	***:13-03-2026***

Task 1: Image Gallery Website

Prompt:

```
Create a responsive Image Gallery Website using only HTML, CSS, and Bootstrap (do NOT use React or any framework).

Requirements:

1. Structure:
- Provide complete project structure (index.html, css/styles.css, optional js/script.js).
- Code should be beginner-friendly and well organized.

2. Layout:
- Use CSS Grid for the gallery layout.
- Use Bootstrap for container, spacing, and responsive utilities.
- Display images in a clean grid layout.

3. Images:
- Use online images from Unsplash or Picsum (no local images).
- Add proper alt text.

4. Styling (Important):
- Make the design colorful using gradients and modern UI styles.
- Add card shadow, rounded corners, and smooth hover effects.

5. Effects:
- Add hover zoom effect on images.
- Add overlay text appearing on hover.

6. Responsiveness:
- Fully responsive for mobile, tablet, and desktop.
- Use auto-fit + minmax for grid responsiveness.

7. Output Format:
- Provide full working code for each file separately.
- Explain briefly how responsiveness works.

Goal:
The gallery should look modern, colorful, and smooth with clean animations.
```

Code Screenshot:

index.html ✕ # styles.css prompt.txt

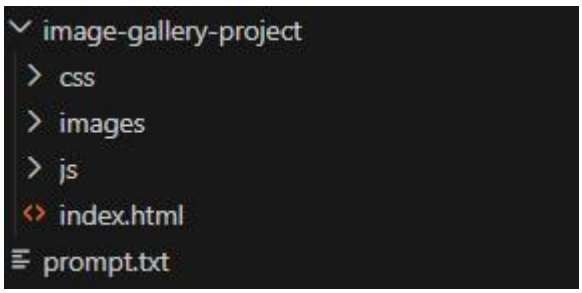
image-gallery-project > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Colorful Image Gallery</title>

<!-- Bootstrap -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css

<link rel="stylesheet" href="css/styles.css">
</head>

<body>

<!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-gradient">
<div class="container">
<a class="navbar-brand fw-bold">🎨 Color Gallery</a>
</div>
</nav>

<!-- Hero -->
<div class="hero text-center text-white">
<h1 class="fw-bold">Modern Responsive Image Gallery</h1>
<p>Bootstrap + CSS Grid + Hover Effects</p>
</div>

<!-- Gallery -->
<div class="container py-5">

<div class="gallery">

<!-- Picsum random -->
<div class="gallery-item">
<img src="https://picsum.photos/600/500?random=1">
<div class="overlay">Creative Design</div>
</div>

<div class="gallery-item">
<img src="https://picsum.photos/600/500?random=2">
<div class="overlay">Workspace</div>
</div>
```

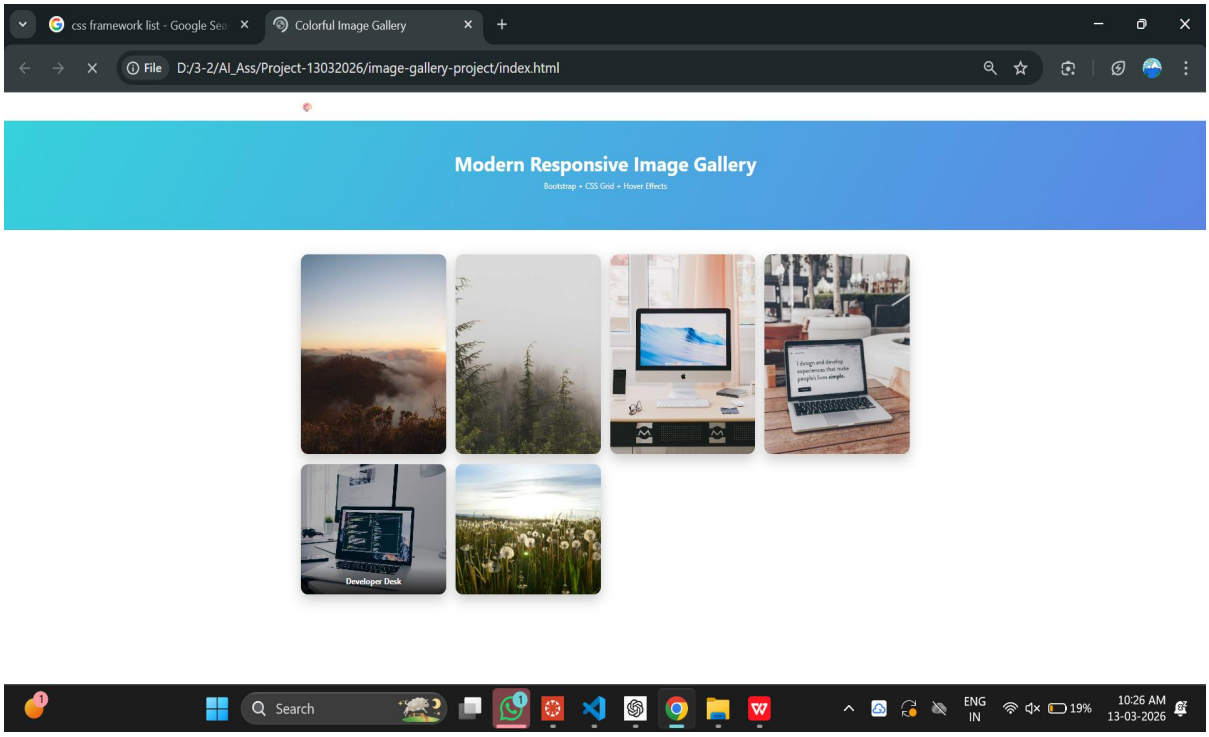
index.html | img1.jpg | # styles.css | prompt.txt

age-gallery-project > css > # styles.css > { } @media (max-width:500px)

```
/* ---------- Navbar Gradient ---------- */

.bg-gradient {
background: linear-gradient(45deg, #ff6a00, #ee0979);
}

/* ---------- Hero Section ---------- */

.hero {
padding: 60px 20px;
background: linear-gradient(120deg, #36d1dc, #5b86e5);
}

/* ---------- Gallery Layout ---------- */

.gallery {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px,1fr));
gap: 20px;
}

/* ---------- Cards ---------- */

.gallery-item {
position: relative;
overflow: hidden;
border-radius: 15px;
box-shadow: 0 10px 25px rgba(0,0,0,0.2);
}

/* Images */

.gallery-item img {
width: 100%;
height: 100%;
object-fit: cover;
transition: transform .4s ease, filter .4s ease;
}

/* Hover Effects */

.gallery-item:hover img {
transform: scale(1.1);
filter: brightness(70%);
```

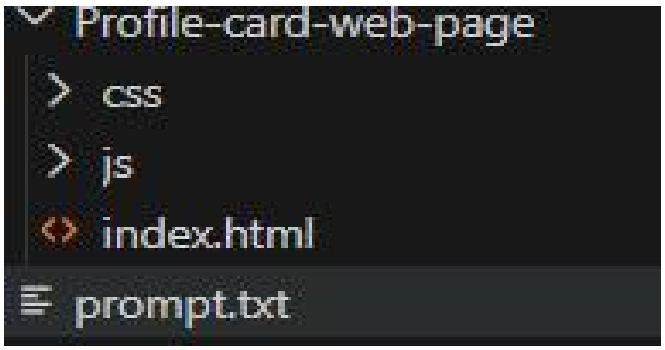


Code Output:



Task 2: Profile Card Generator.

Structure:



Prompt:

prompt.txt

```
Create a modern Profile Card web page using only HTML, CSS, and Bootstrap (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, optional js/script.js).
- Keep code beginner-friendly and clean.

2. Card Content:
- Show user image, name, role, short bio, and social media links.

3. Layout:
- Use Flexbox to design the profile card layout.
- Center the card vertically and horizontally.

4. Styling:
- Suggest modern UI colors and Google Fonts.
- Use gradients, rounded corners, and soft shadows.

5. Effects:
- Add hover animation on card.
- Add hover effects on social icons.

6. Responsiveness:
- Fully responsive for mobile and desktop.

7. Output:
- Provide full working code for each file separately.
```

Code Screenshot:

index.html | styles.css | prompt.txt

Profile-card-web-page > index.html > html

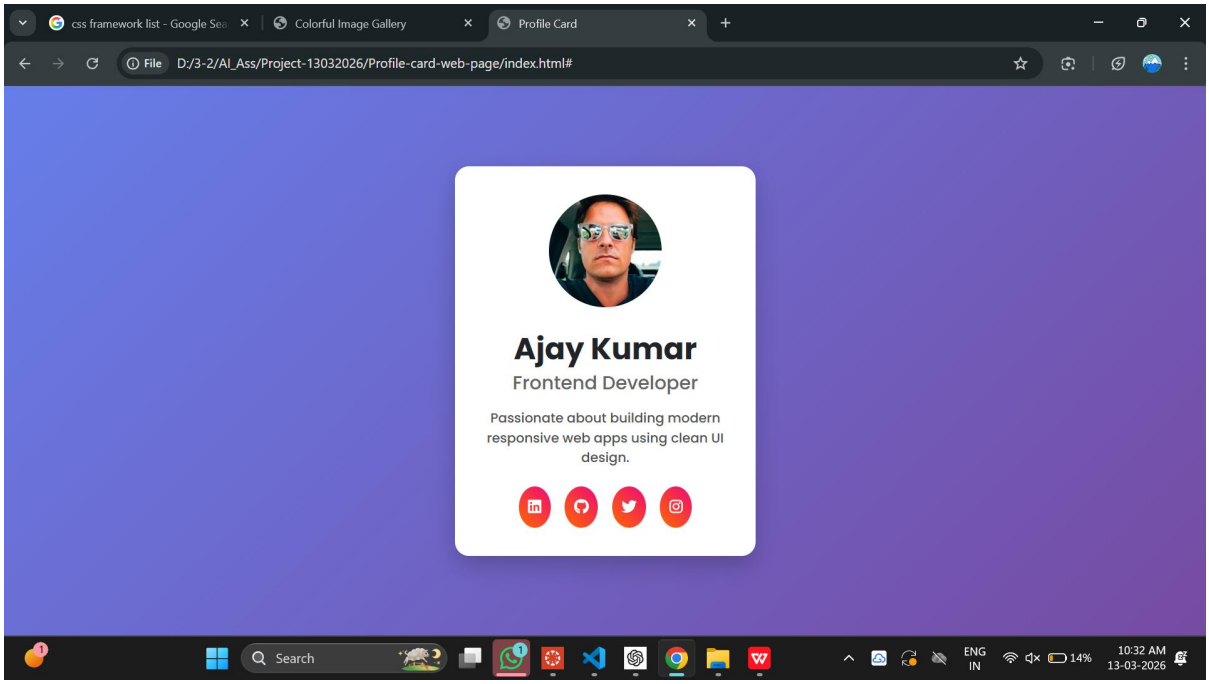
```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Profile Card</title>

<!-- Bootstrap -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">

<!-- Google Font -->
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;500;700&display=swap" rel="stylesheet">

<!-- Icons -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">

<link rel="stylesheet" href="css/styles.css">
</head>

<body>

<div class="main-container">

<div class="profile-card">

<img src="https://i.pravatar.cc/300" class="profile-img" alt="profile">

<h2>Ajay Kumar</h2>
<h5>Frontend Developer</h5>

<p class="bio">
Passionate about building modern responsive web apps using clean UI design.
</p>

<div class="social-icons">

<a href="#"><i class="fab fa-linkedin"></i></a>
<a href="#"><i class="fab fa-github"></i></a>
<a href="#"><i class="fab fa-twitter"></i></a>
<a href="#"><i class="fab fa-instagram"></i></a>

</div>
```

index.html | styles.css | prompt.txt

Profile-card-web-page > css > styles.css > .social-icons a:hover

```css
/* ---------- Global ---------- */

body {
margin: 0;
font-family: 'Poppins', sans-serif;

background: linear-gradient(135deg, #667eea, #764ba2);
height: 100vh;
}

/* ---------- Center Layout (Flexbox) ---------- */

.main-container {
display: flex;
justify-content: center;
align-items: center;
height: 100%;
}

/* ---------- Card ---------- */

.profile-card {
background: white;
padding: 30px;
width: 320px;
border-radius: 15px;
text-align: center;

box-shadow: 0 10px 25px rgba(0,0,0,0.2);
transition: .4s;
}

/* Hover Animation */

.profile-card:hover {
transform: translateY(-10px);
box-shadow: 0 15px 35px rgba(0,0,0,0.3);
}

/* ---------- Image ---------- */

.profile-img {
width: 120px;
height: 120px;
```

Output Screenshot:



Task 3: Develop a dynamic shopping cart system.

Prompt:

prompt.txt

```
Create a dynamic shopping cart system using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, js/script.js).
- Code should be beginner-friendly and clean.

2. Features:
- Display product cards with image, name, price, and Add to Cart button.
- Allow users to add, remove, and update product quantities.

3. Logic (Important):
- Manage cart state using JavaScript objects.
- Update total price dynamically in real time.
- Persist cart data using localStorage.

4. UI:
- Use Bootstrap for layout and responsiveness.
- Add hover effects and clean modern styling.

5. Output:
- Provide full working code for each file separately.
```

Code Screenshot:

index.html × | styles.css | prompt.txt

Profile-card-web-page > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Shopping Cart</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
<link rel="stylesheet" href="css/styles.css">
</head>

<body>

<div class="container py-5">

<h2 class="text-center mb-4">🛒 Dynamic Shopping Cart</h2>

<div class="row" id="product-list"></div>

<hr>

<h4>Cart</h4>
<div id="cart-items"></div>

<h3 class="mt-3">Total: ₹<span id="total">0</span></h3>

</div>

<script src="js/script.js"></script>
</body>
</html>
```

index.html | styles.css × | prompt.txt

Profile-card-web-page > css > styles.css > body

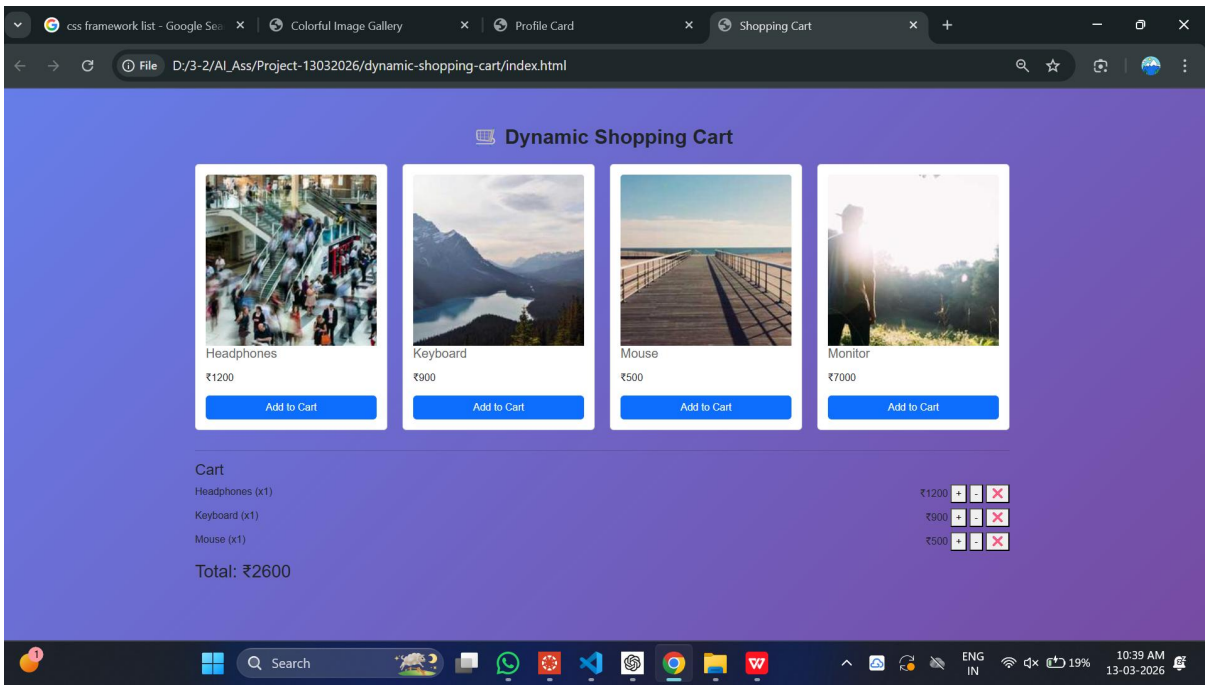
```css
/* ---------- Global ---------- */

body {
margin: 0;
font-family: 'Poppins', sans-serif;

background: linear-gradient(135deg, #667eea, #764ba2);
height: 100vh;
}

/* ---------- Center Layout (Flexbox) ---------- */

.main-container {
display: flex;
justify-content: center;
align-items: center;
height: 100%;
}

/* ---------- Card ---------- */

.profile-card {
background: white;
padding: 30px;
width: 320px;
border-radius: 15px;
text-align: center;

box-shadow: 0 10px 25px rgba(0,0,0,0.2);
transition: .4s;
}

/* Hover Animation */

.profile-card:hover {
transform: translateY(-10px);
box-shadow: 0 15px 35px rgba(0,0,0,0.3);
}

/* ---------- Image ---------- */

.profile-img {
width: 120px;
height: 120px;
```

index.html | styles.css | prompt.txt | script.js

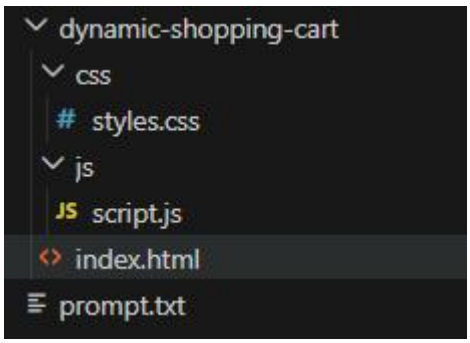
dynamic-shopping-cart > js > script.js > ...

```javascript
function renderProducts(){
products.forEach(p=>{
<div class="card p-3 mb-3">
<img src="${p.img}" class="card-img-top">
<h5>${p.name}</h5>
<p>₹${p.price}</p>
<button class="btn btn-primary" onclick="addToCart(${p.id})">Add to Cart</button>
</div>
</div>
`;
});
}

// -------- Add to Cart --------

function addToCart(id){
cart[id]=cart[id]?cart[id]+1:1;
updateCart();
}

// -------- Update Cart UI --------

function updateCart(){
localStorage.setItem("cart",JSON.stringify(cart));

const cartDiv=document.getElementById("cart-items");
cartDiv.innerHTML="";
let total=0;

for(let id in cart){
let product=products.find(p=>p.id==id);
let qty=cart[id];
let subtotal=product.price*qty;

total+=subtotal;

cartDiv.innerHTML+=`
<div class="d-flex justify-content-between mb-2">
<span>${product.name} (x${qty})</span>
<span>
₹${subtotal}
<button onclick="changeQty(${id},1)">+</button>
<button onclick="changeQty(${id},-1)">-</button>
<button onclick="removeItem(${id})">❌</button>
```

Output Screenshot:

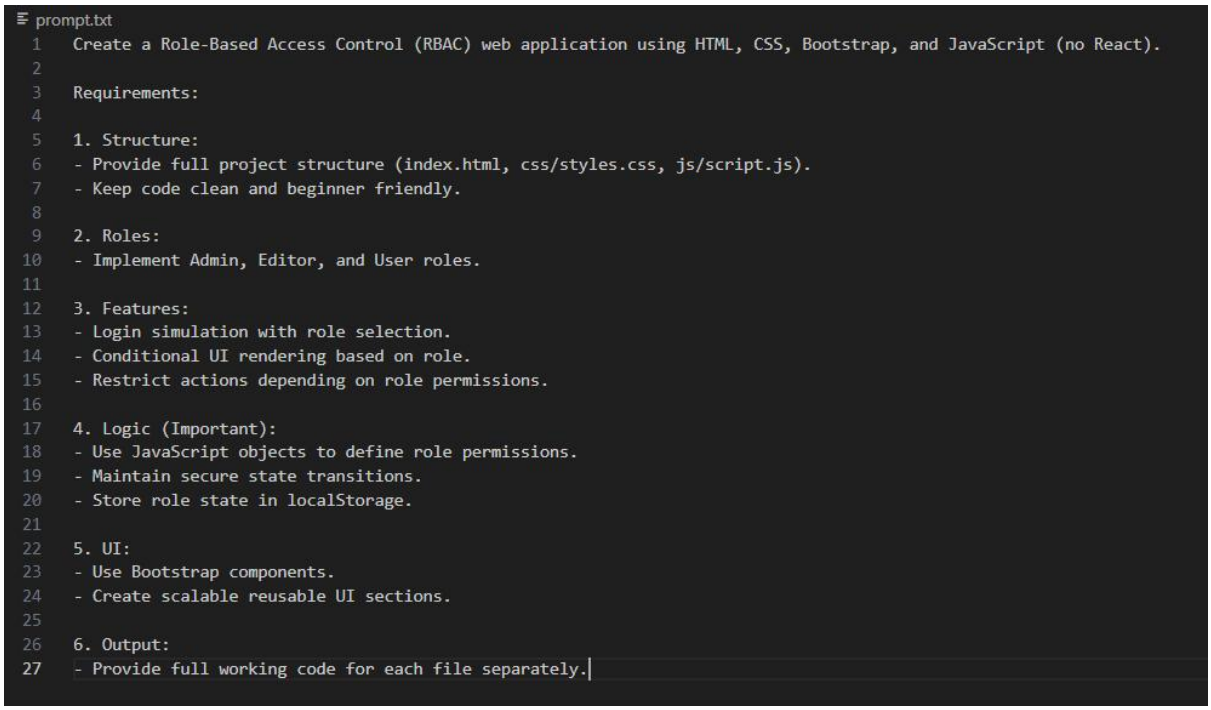


Structure:



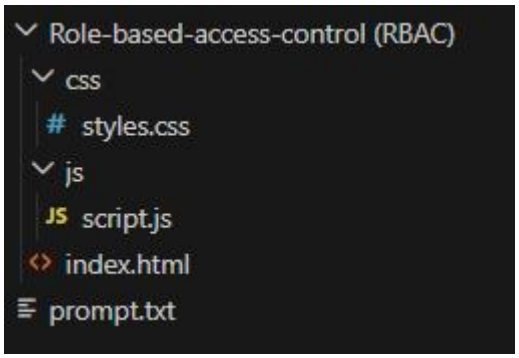
Task 4: Role-Based Access Control Application.

Prompt:



```
prompt.txt
Create a Role-Based Access Control (RBAC) web application using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, js/script.js).
- Keep code clean and beginner friendly.

2. Roles:
- Implement Admin, Editor, and User roles.

3. Features:
- Login simulation with role selection.
- Conditional UI rendering based on role.
- Restrict actions depending on role permissions.

4. Logic (Important):
- Use JavaScript objects to define role permissions.
- Maintain secure state transitions.
- Store role state in localStorage.

5. UI:
- Use Bootstrap components.
- Create scalable reusable UI sections.

6. Output:
- Provide full working code for each file separately.
```

Structure:



SR UNIVERSITY

Code Screenshot:

index.html | styles.css | prompt.txt | script.js

Role-based-access-control (RBAC) > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>RBAC App</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
<link rel="stylesheet" href="css/styles.css">
</head>

<body>

<div class="container py-5">

<h2 class="text-center mb-4">🔐 Role-Based Access Control</h2>

<!-- Login Simulation -->
<div class="text-center mb-4">
<select id="roleSelect" class="form-select w-25 mx-auto">
<option value="">Select Role</option>
<option value="admin">Admin</option>
<option value="editor">Editor</option>
<option value="user">User</option>
</select>

<button class="btn btn-primary mt-2" onclick="login()">Login</button>
<button class="btn btn-danger mt-2" onclick="logout()">Logout</button>
</div>

<hr>

<!-- Dashboard -->
<div id="dashboard"></div>

</div>

<script src="js/script.js"></script>
</body>
</html>
```

index.html | styles.css | prompt.txt | script.js

Role-based-access-control (RBAC) > css > styles.css > .card:hover

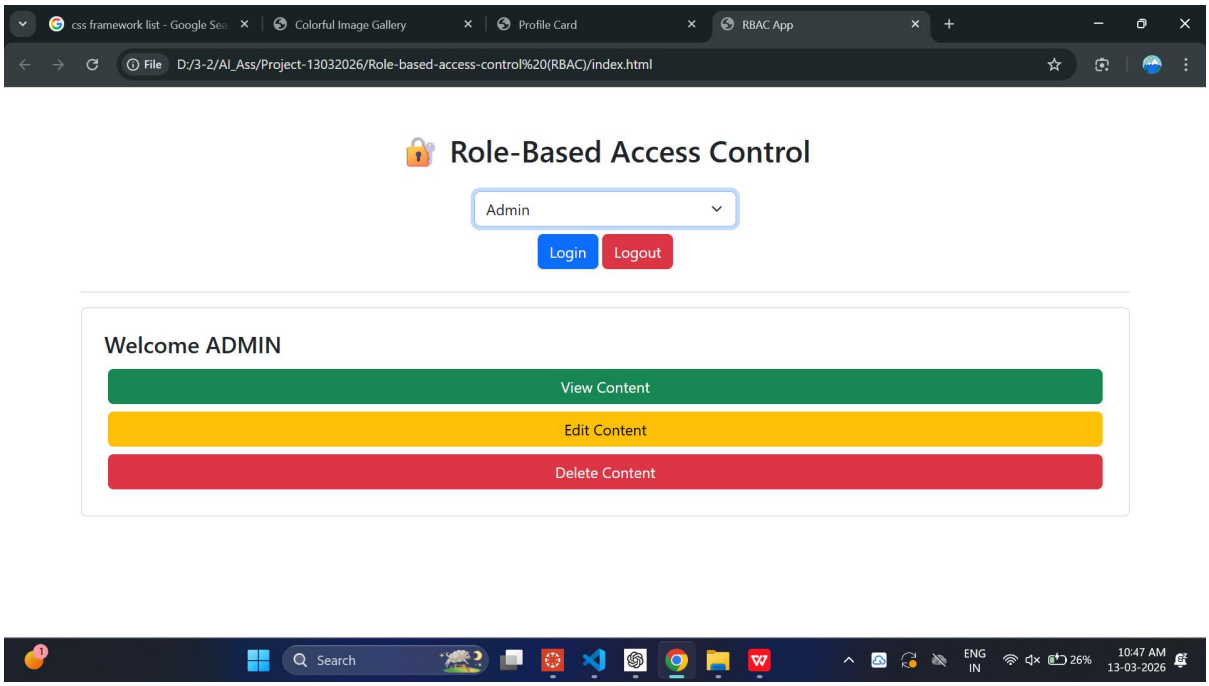
```css
.card{
transition:.3s;
}

.card:hover{
transform: translateY(-5px);
}
```

index.html | styles.css | prompt.txt | script.js

Role-based-access-control (RBAC) > js > script.js > ...

```javascript
// -------- Role Permissions --------

const roles = {
admin: ["view","edit","delete"],
editor: ["view","edit"],
user: ["view"]
};

// -------- State --------

let currentRole = localStorage.getItem("role") || null;

// -------- Login --------

function login(){
const role=document.getElementById("roleSelect").value;
if(!role) return alert("Select role");

currentRole=role;
localStorage.setItem("role",role);
renderUI();
}

// -------- Logout --------

function logout(){
localStorage.removeItem("role");
currentRole=null;
renderUI();
}

// -------- Conditional Rendering --------

function renderUI(){
const dash=document.getElementById("dashboard");

if(!currentRole){
dash.innerHTML="<p class='text-center'>Please login</p>";
return;
}

let perms=roles[currentRole];

dash.innerHTML=`
```

Code Output:



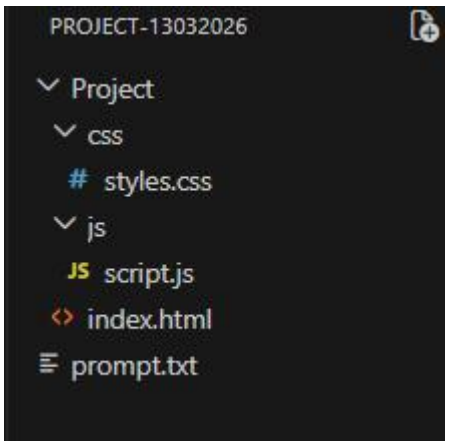
Task 5: Multi-Language Website

Prompt:

prompt.txt

```
Create a multilingual website using HTML, CSS, Bootstrap, and JavaScript (no React).

Requirements:

1. Structure:
- Provide full project structure (index.html, css/styles.css, js/script.js).
- Keep code beginner-friendly and clean.

2. Features:
- Dynamically switch languages (English, Hindi, Arabic).
- Use structured translation files (JavaScript objects).

3. Logic (Important):
- Persist selected language using localStorage.
- Automatically apply saved language on reload.

4. Layout:
- Support both LTR and RTL layouts.
- Switch direction dynamically when Arabic is selected.

5. Accessibility:
- Use semantic HTML.
- Add aria-labels where needed.

6. Output:
- Provide full working code for each file separately.
```

Structure:



Code Screenshot:

index.html | styles.css | prompt.txt | script.js

Role-based-access-control (RBAC) > index.html > html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Multi Language Website</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
<link rel="stylesheet" href="css/styles.css">
</head>

<body>

<div class="container py-5">

<h2 id="title">Multi Language Website</h2>

<!-- Language Switch -->
<select id="langSelect" class="form-select w-25 mb-4" aria-label="Language Switcher">
<option value="en">English</option>
<option value="hi">Hindi</option>
<option value="ar">Arabic</option>
</select>

<p id="desc"></p>

<button class="btn btn-primary" id="btn"></button>

</div>

<script src="js/script.js"></script>
</body>
</html>
```

Project > css > styles.css > body

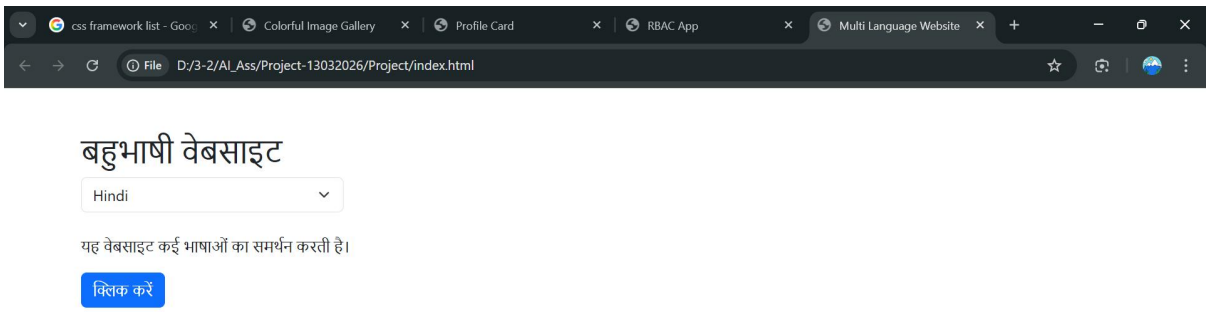
```css
body{
transition:.3s;
}
```

index.html | styles.css | prompt.txt | script.js

Project > js > script.js > ...

```javascript
// -------- Translations --------

const translations = {

en:{
title:"Multi Language Website",
desc:"This website supports multiple languages dynamically.",
btn:"Click Me"
},

hi:{
title:"बहुभाषी वेबसाइट",
desc:"यह वेबसाइट कई भाषाओं का समर्थन करती है।",
btn:"क्लिक करें"
},

ar:{
title:"موقع متعدد اللغات",
desc:"هذا الموقع يدعم لغات متعددة.",
btn:"اضغط هنا"
}

};

// -------- Change Language --------

function setLanguage(lang){

localStorage.setItem("lang",lang);

document.getElementById("title").innerText=translations[lang].title;
document.getElementById("desc").innerText=translations[lang].desc;
document.getElementById("btn").innerText=translations[lang].btn;

// RTL Support
document.body.dir=(lang=="ar")?"rtl":"ltr";
}

// -------- Event --------

document.getElementById("langSelect").addEventListener("change",(e)=>{
setLanguage(e.target.value);
});
```

Code Output:



SR UNIVERSITY

Task 5: Online Examination System

Prompt:

```
prompt.txt
1. Analyze student performance and give suggestions.
2. Detect suspicious behavior (tab switching/inactivity).
3. Improve UX with reminders.
4. Simplify questions for accessibility.
```

Structure:

```
PROJECT-13032026
Project
  css
    styles.css
  js
    script.js
  index.html
  prompt.txt
```

Code Screenshot:

```
index.html   styles.css   prompt.txt   script.js
Project > index.html > html
<html lang="en">
<body>
<div class="container">
<form id="examForm">
<div class="question">
<label><input type="radio" name="q1" value="O(n)"> O(n)</label>
<label><input type="radio" name="q1" value="O(log n)"> O(log n)</label>
<label><input type="radio" name="q1" value="O(n log n)"> O(n log n)</label>

</div>

<div class="question">
<p>2. Which data structure uses FIFO?</p>

<label><input type="radio" name="q2" value="Stack"> Stack</label>
<label><input type="radio" name="q2" value="Queue"> Queue</label>
<label><input type="radio" name="q2" value="Tree"> Tree</label>

</div>

<button type="submit">Submit Exam</button>

</form>

<div id="result"></div>

</div>

<script src="js/script.js"></script>
</body>
</html>
```

index.html | styles.css | prompt.txt | script.js

Project > css > styles.css > #result

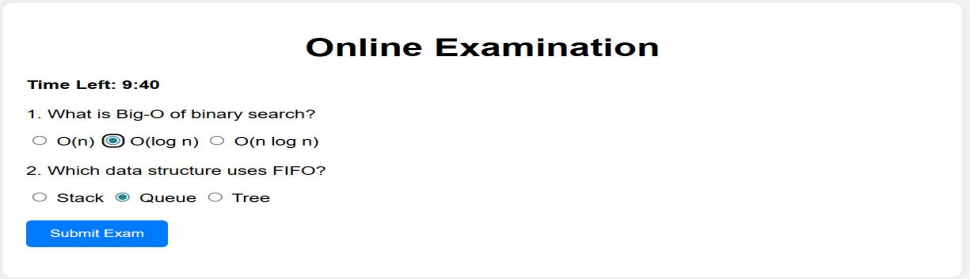
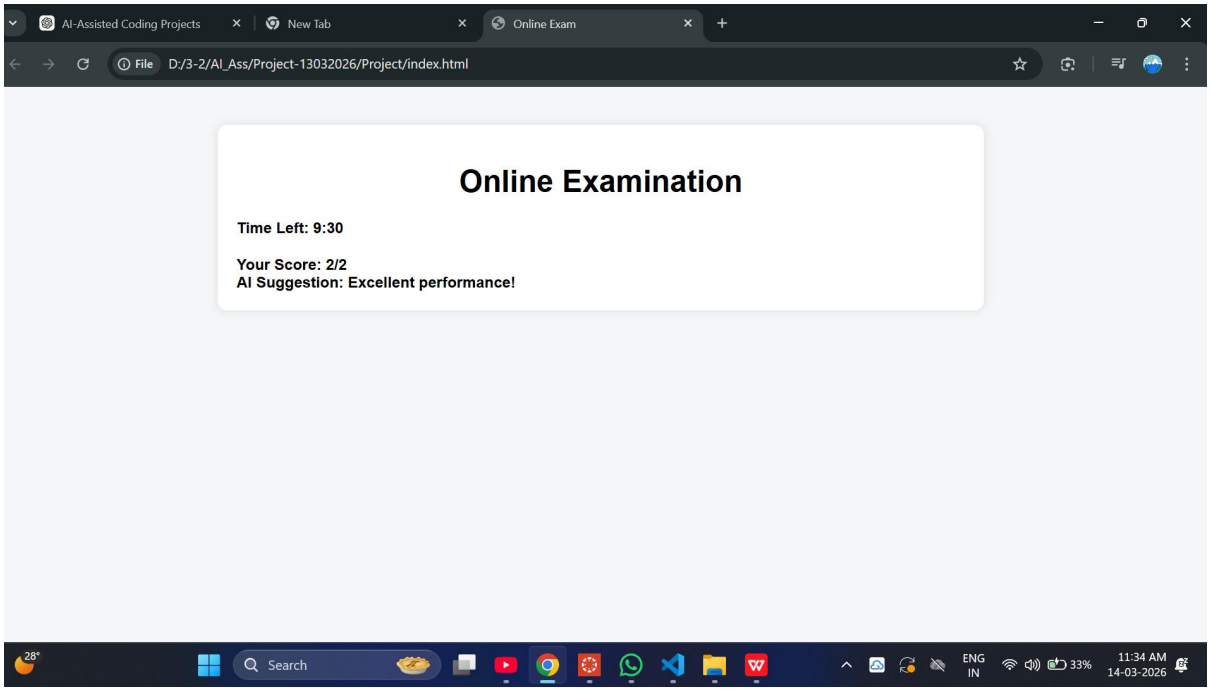
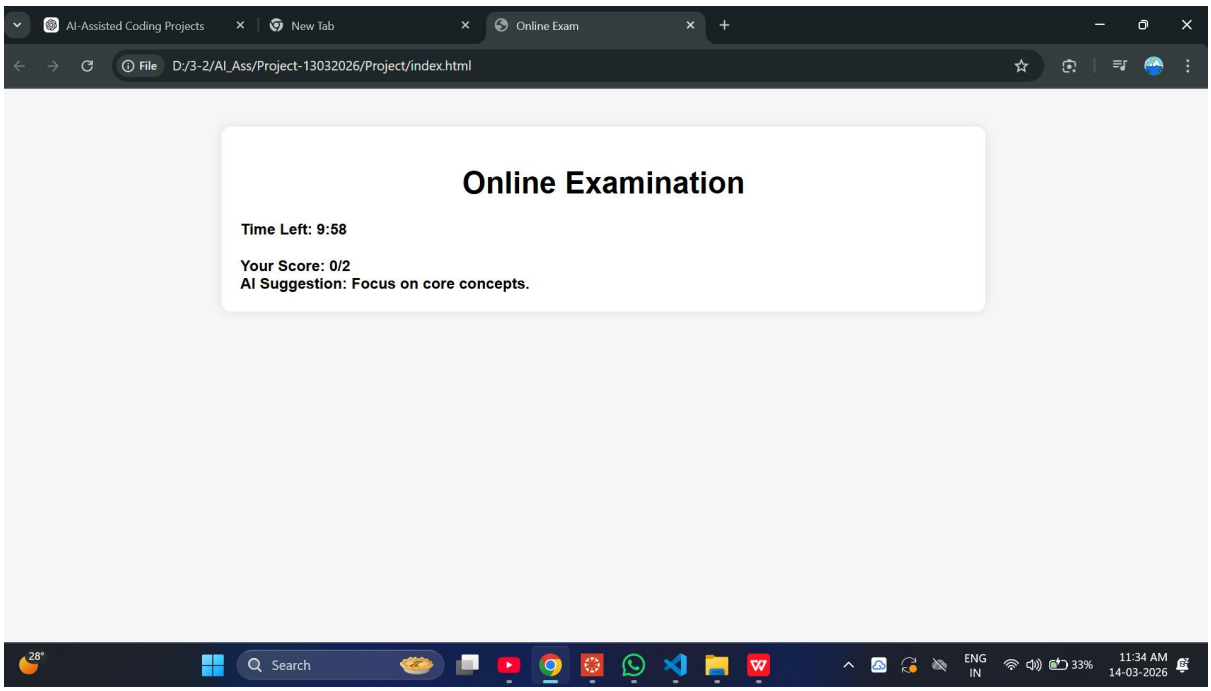
```
body {
font-family: Arial, sans-serif;
background: #f4f6f8;
margin: 0;
padding: 0;
}

.container {
width: 60%;
margin: auto;
margin-top: 40px;
background: white;
padding: 20px;
border-radius: 10px;
box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
}

h1 {
text-align: center;
}

.top-bar {
display: flex;
justify-content: space-between;
margin-bottom: 20px;
font-weight: bold;
}

.question {
margin-bottom: 20px;
}

button {
padding: 10px 20px;
border: none;
background: #007bff;
```

index.html | styles.css | prompt.txt | script.js

Project > js > script.js > ...

```javascript
function autoSubmit() {
alert("Time is up! Auto submitting.");
submitExam();
}

// ================= AI FEATURES =================

// AI UX Improvement (simple simulation)
function runAIUX(score) {

let aiMessage = "";

if (score === 2)
aiMessage = "AI Suggestion: Excellent performance!";
else if (score === 1)
aiMessage = "AI Suggestion: Revise DSA basics.";
else
aiMessage = "AI Suggestion: Focus on core concepts.";

resultDiv.innerHTML += `<br>${aiMessage}`;
}

// ================= TAB SWITCH MONITOR =================

document.addEventListener("visibilitychange", () => {

if (document.hidden) {
alert("Warning: Tab switching detected!");
}

});
```

Code Output:



SR UNIVERSITY